

Web Server Lab

My Lab environment:

```
brishtibasu@Brishtis-Air ~ % ifconfig
lo0: flags=8049<UP,LOOPBACK,RUNNING,MULTICAST> mtu 16384
options=1203<RXCSUM, TXCSUM, TXSTATUS, SW_TIMESTAMP>
inet 127.0.0.1 netmask 0xff000000
inet6 ::1 prefixlen 128
inet6 fe80::1%lo0 prefixlen 64 scopeid 0x1
nd6 options=201<PERFORMNUD,DAD>
gif0: flags=8010<POINTOPOINT,MULTICAST> mtu 1280
stf0: flags=0<> mtu 1280
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
options=400<CHANNEL_IO>
ether 14:c2:13:0f:e5:74
inet6 fe80::1464:a10:9471:c578%en0 prefixlen 64 secured scopeid 0x4
inet 192.168.1.160 netmask 0xfffff000 broadcast 192.168.1.255
inet6 2600:4041:44ca:8800:8a6:3b67:7c21:62a0 prefixlen 64 autoconf secured
inet6 2600:4041:44ca:8800:eda1:63d4:fd34:b8b8 prefixlen 64 autoconf temporary
nd6 options=201<PERFORMNUD,DAD>
media: autoselect
status: active
en1: flags=8963<UP,BROADCAST,SMART,RUNNING,PROMISC,SIMPLEX,MULTICAST> mtu 1500
options=460<TSO4,TSO6,CHANNEL_IO>
ether 82:18:5d:7f:5c:40
media: autoselect <full-duplex>
status: inactive
bridge0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
options=63<RXCSUM, TXCSUM, TSO4, TSO6>
ether 82:18:5d:7f:5c:40
Configuration:
id 0:0:0:0:0:0 priority 0 hellotime 0 fwddelay 0
maxage 0 holdcnt 0 proto stp maxaddr 100 timeout 1200
root id 0:0:0:0:0:0 priority 0 ifcost 0 port 0
ipfilter disabled flags 0x0
member: en1 flags=3<LEARNING,DISCOVER>
ifmaxaddr 0 port 5 priority 0 path cost 0
nd6 options=201<PERFORMNUD,DAD>
media: <unknown type>
status: inactive
p2p0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> mtu 2304
```

Line by Line explanation of the code:

- **First, we created and configured the server socket:**

```
serverSocket = socket(AF_INET, SOCK_STREAM) # Prepare a server socket
serverSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
```

- `socket(AF_INET, SOCK_STREAM)`: Creates a **TCP socket** (stream-oriented, IPv4).
- `setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)`: Allows the port to be reused immediately after the server stops, preventing the “Address already in use” error when restarting.

- **Binding and listening**

```
serverSocket.bind("", port))  
serverSocket.listen(1)
```

- `bind("", port)` tells the socket to listen on the given port (e.g., 6789) for all interfaces.
- `listen(1)` makes the socket a listening socket and sets the backlog to 1 (how many pending connections can wait in the queue).

- **Accepting the connection:**

```
connectionSocket, addr = serverSocket.accept()
```

- Blocks until a client connects; returns a dedicated socket and client address.

- **Receive Request and Parse Filename & Read File**

```
message = connectionSocket.recv(1024).decode()
```

```
if not message:  
    connectionSocket.close()  
    continue
```

```
request_line = message.splitlines()[0]  
print(f'Request: {request_line}')
```

```
filename = message.split()[1]
```

- `recv(1024)`: Reads up to 1024 bytes from the client request.
- `.decode()`: Converts bytes into a string.
- If no data is received, the server closes the connection and waits for another request.
- `message.splitlines()[0]`: Extracts the first line of the request (e.g., GET /HelloWorld.html HTTP/1.1).
- `filename = message.split()[1]`: Extracts the requested path (e.g., /HelloWorld.html).

- **Opening and reading the requested file:**

```
f = open(filename[1:])           # remove leading "/"  
outputdata = f.read()
```

- filename[1:]: Removes the leading / so it matches a local file in the same directory (e.g., /HelloWorld.html → HelloWorld.html).
- Reads the file contents into the variable outputdata.
- **Sending a 200 OK response:**

```
connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
print("Responded with: 200 OK")
```

 - Sends an HTTP header line indicating success (200 OK).
 - \r\n\r\n: Marks the end of the headers.
 - Prints confirmation for the server log.
- **Handling missing files (404 Not Found):**

```
connectionSocket.send("HTTP/1.1 404 Not Found\r\n\r\n".encode())
connectionSocket.send("<html><body><h1>404 Not
Found</h1></body></html>".encode())
print("Responded with: 404 Not Found")
```

 - If the file does not exist, the server sends an HTTP response with status **404 Not Found**.
 - A simple HTML body is sent to show a friendly error page.
 - Prints confirmation for the server log.
- **Closing the client socket:**

```
connectionSocket.close()
```

 - After serving the request (200 or 404), the connection to the client is closed.
 - The loop restarts and the server is ready for the next request.

I also used the following robustness line in my testing version:

serverSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1): This allows immediate reuse of the port upon restart, avoiding “Address already in use” errors.

Screenshots of output:

```
brishtibas@10-20-28-246 cn_socket % python -u "/Users/brishtibas/Downloads/cn_socket/server.py"
zsh: command not found: python
brishtibas@10-20-28-246 cn_socket % python3 -u "/Users/brishtibas/Downloads/cn_socket/server.py"
Ready to serve...
Connection established with ('127.0.0.1', 52597)
Request: GET /HelloWorld.html HTTP/1.1
Responded with: 200 OK
Ready to serve...
Connection established with ('127.0.0.1', 52598)
Request: GET /random.html HTTP/1.1
Responded with: 404 Not Found
Ready to serve...
Connection established with ('127.0.0.1', 52608)
```



Hello World!

Web page found



404 Not Found